Aim: To create a basic blockchain using Python by implementing blocks, cryptographic hashing, transactions, Proof of Work, mining, and blockchain validation.

Software Requirements:

- Python 3.x
- Google Colab / Jupyter Notebook / VS Code
- Python libraries:
    - `hashlib`
    - `json`
    - `time`

Theory:A **blockchain** is a distributed digital ledger that stores data in a sequence of interconnected blocks. Each block contains information such as transaction data, a timestamp, a unique hash, and the hash of the previous block.

The connection between blocks is created using cryptographic hashing. In this experiment, the **SHA-256** hashing algorithm is used. If any information inside a block is changed, its hash also changes, making it possible to detect tampering.

The first block of a blockchain is called the **Genesis Block**. Subsequent blocks contain the hash of the previous block, creating a chain.

Main Components:

1. Block:A block stores transaction data along with its index, timestamp, previous hash, nonce, and current hash.

2. Hash:A hash is a fixed-length string generated from the contents of a block. SHA-256 is used to generate the hash.

3. Previous Hash: Each block stores the hash of the previous block. This links all blocks together.

4. Nonce: Nonce is a number that is changed during mining until a valid hash satisfying the required difficulty is found.

5. Proof of Work:
Proof of Work requires the miner to perform computational work to find a valid block hash. In this experiment, the hash must start with a specified number of zeros.

6. Mining:
Mining is the process of finding a valid nonce for a block using Proof of Work and adding the block to the blockchain.

7. Blockchain Validation:
The blockchain is checked by recalculating each block's hash and verifying that the previous hash stored in every block matches the actual hash of the preceding block.

CODE AND OUTPUT:

Step 1 — Import required Libraries

[1]
✓ 0s

```
import hashlib
import json
from time import time

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

Step 2 — Create the `Block` Class

```
class Block:
   def __init__(self, index, timestamp, transactions, previous_hash,
nonce=0):
        self.index = index
        self.timestamp = timestamp
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.nonce = nonce
        self.hash = self.calculate_hash()

   def calculate_hash(self):
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "transactions": self.transactions,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce
```

```
        }, sort_keys=True).encode()

        return hashlib.sha256(block_string).hexdigest()

    def mine_block(self, difficulty):
        target = "0" * difficulty

        while self.hash[:difficulty] != target:
            self.nonce += 1
            self.hash = self.calculate_hash()

        print("Block mined:", self.hash)
```

Step 3 — Create the `blockchain` Class

```
class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]
        self.pending_transactions = []
        self.difficulty = 4
        self.mining_reward = 10

    def create_genesis_block(self):
        return Block(0, time(), [], "0")

    def get_latest_block(self):
        return self.chain[-1]

    def add_transaction(self, sender, receiver, amount):
        self.pending_transactions.append({
            "sender": sender,
            "receiver": receiver,
            "amount": amount
        })

    def mine_pending_transactions(self, miner_address):
        self.pending_transactions.append({
            "sender": "SYSTEM",
            "receiver": miner_address,
            "amount": self.mining_reward
        })
```

```
        block = Block(
            len(self.chain),
            time(),
            self.pending_transactions,
            self.get_latest_block().hash
        )

        block.mine_block(self.difficulty)

        self.chain.append(block)
        self.pending_transactions = []

    def is_chain_valid(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i - 1]

            if current.hash != current.calculate_hash():
                return False

            if current.previous_hash != previous.hash:
                return False

        return True
```

Step 4 — Create the Blockchain and Genesis Block

```
blockchain = Blockchain()

print("Genesis Block Created Successfully!")
print("Block Index:", blockchain.chain[0].index)
print("Genesis Block Hash:", blockchain.chain[0].hash)
print("Previous Hash:", blockchain.chain[0].previous_hash)
```

```
Genesis Block Created Successfully!
Block Index: 0
Genesis Block Hash: 49e900d1541efc35aa1bdd2b46ce9afac55a666f4381cb8b5e066ebe8482285d
Previous Hash: 0
```

Step 5 — Add Transactions

```
blockchain.add_transaction("Deepak", "Adwani", 25)
blockchain.add_transaction("Dhiraj", "Adani", 10)

print("Transactions added successfully!")
print(blockchain.pending_transactions)
```

```
...
    {'sender': 'deepak', 'receiver': 'Advani', 'amount': 25}, {'sender': 'dhiraj', 'receiver
```

Step 6 — Mine the First Block

```
print("Mining Block 1...")

blockchain.mine_pending_transactions("Miner1")
```

```
...  Mining Block 1...
     Block mined: 000026bdef37996095c1c5fb2511cf1cb83bf2e2d158c749a24320fd41a6a932
```

Step 7 — Add Another Transaction

```
blockchain.add_transaction("deep", "vesit", 5)

print("New transaction added successfully!")
print(blockchain.pending_transactions)
```

```
...
    5}, {'sender': 'deep', 'receiver': 'vesit', 'amount': 5}, {'sender': 'deep', 'receiver':
```

Step 8 — Mine the Second Block

```
print("Mining Block 2...")

blockchain.mine_pending_transactions("Miner1")
```

```
...  Mining Block 2...
     Block mined: 00009b601034a5804b1dda015703037c8cb5ae7e5226fe033324b2ede0d1bdba
```

Step 9 — Display the Complete Blockchain

```
for block in blockchain.chain:
    print("\n==============================")
    print("Block Index:", block.index)
    print("Timestamp:", block.timestamp)
    print("Transactions:", block.transactions)
    print("Previous Hash:", block.previous_hash)
    print("Hash:", block.hash)
    print("Nonce:", block.nonce)
```

```
...
    ==============================
    Block Index: 0
    Timestamp: 1788330755.6601574
    Transactions: []
    Previous Hash: 0
    Hash: 49e900d1541efc35aa1bdd2b46ce9afac55a666f4381cb8b5e066ebe8482285d
    Nonce: 0

    ==============================
    Block Index: 1
    Timestamp: 1788331369.1914725
    Transactions: [{'sender': 'Alice', 'receiver': 'Bob', 'amount': 25}, {'sender': 'Bob', '
    Previous Hash: 49e900d1541efc35aa1bdd2b46ce9afac55a666f4381cb8b5e066ebe8482285d
    Hash: 000026bdef37996095c1c5fb2511cf1cb83bf2e2d158c749a24320fd41a6a932
    Nonce: 12291

    ==============================
    Block Index: 2
    Timestamp: 1788331696.8241503
    Transactions: [{'sender': 'Charlie', 'receiver': 'David', 'amount': 5}, {'sender': 'deep
    Previous Hash: 000026bdef37996095c1c5fb2511cf1cb83bf2e2d158c749a24320fd41a6a932
    Hash: 00009b601034a5804b1dda015703037c8cb5ae7e5226fe033324b2ede0d1bdba
    Nonce: 16689
```

Step 10 — Validate the Blockchain

```
print("Checking Blockchain Integrity...")
print("Is Blockchain Valid?", blockchain.is_chain_valid())
```

```
...  Checking Blockchain Integrity...
    Is Blockchain Valid? True
```

Step 11 — Test Blockchain Tampering

```
# Tampering with Block 1
blockchain.chain[1].transactions[0]["amount"] = 1000

print("Transaction modified!")
print("Is Blockchain Valid?", blockchain.is_chain_valid())
```

```
...  Transaction modified!
    Is Blockchain Valid? False
```

Conclusion:A basic blockchain was successfully implemented using Python. The experiment demonstrated block creation, transactions, cryptographic hashing, Proof of Work mining, and blockchain validation. The tampering test showed that changes in block data can be detected using hashes.